

Evolução Arquitetural: Servidor HTTP Simplificado sobre R-UDP/TCP e Resolução de Nomes Local (DNS)

Julio Cesar de Lima Mendes

¹ Campus Senador Helvídio Nunes de Barros – Universidade Federal do Piauí (UFPI)
Curso de Bacharelado em Sistemas de Informação – Redes de Computadores II
Matrícula: 20249006910

juliocesarlimendes@gmail.com

Abstract. *This paper presents an experimental analysis of an architectural evolution integrating a local Domain Name System (DNS) resolution module and a simplified HTTP/1.1 Web Server application layer. The system operates alternately over native TCP and a custom Reliable UDP (R-UDP) transport protocol based on a Stop-and-Wait flow control mechanism. Using an expanded Docker container environment and tc/netem network emulation, the performance and reliability of both approaches were evaluated across three file sizes (100KB, 500KB, and 1MB) under three network scenarios: ideal (0% loss / 10ms delay), degraded (5% loss / 50ms), and high-loss (10% loss / 100ms). The results demonstrated that while both transport stacks successfully guaranteed 100% data integrity and operational correctness, HTTP-RUDP experienced severe throughput limitations (averaging around 0.0384 to 0.160 Mbps) due to initial socket buffer synchronization mismatches and the sifting of duplicate requests under static timeouts. Traffic validation using packet captures (.pcap) confirmed full architectural alignment and the persistent inclusion of the X-Custom-Auth token.*

Resumo. *Este artigo apresenta uma análise experimental de uma evolução arquitetural que integra um módulo de resolução de nomes local (DNS) e uma camada de aplicação de Servidor Web HTTP/1.1 simplificado. O sistema opera alternadamente sobre o protocolo TCP nativo e um protocolo personalizado de UDP Confiável (R-UDP) baseado no mecanismo de controle de fluxo Stop-and-Wait. Utilizando um ambiente expandido de containers Docker e emulação de rede com tc/netem, avaliou-se o desempenho e a confiabilidade de ambas as abordagens para três tamanhos de arquivo (100KB, 500KB e 1MB) sob três cenários de rede: ideal (0% de perda / 10ms de delay), degradada (5% de perda / 50ms) e alta perda (10% de perda / 100ms). Os resultados demonstram que, embora ambas as pilhas tenham garantido 100% de integridade e correção funcional, o HTTP-RUDP sofreu severas restrições de vazão (throughput médio entre 0,0384 e 0,160 Mbps) devido a desalinhamentos iniciais de sincronização no buffer do socket e ao descarte de requisições duplicadas sob timeouts estáticos. A validação do tráfego via capturas de pacotes (.pcap) confirmou o alinhamento arquitetural e a inclusão persistente do token X-Custom-Auth.*

1. Introdução

A arquitetura da Internet é fundamentada em uma divisão clara de camadas, onde os protocolos de transporte TCP e UDP oferecem diferentes garantias de serviço, servindo

de base para protocolos essenciais da camada de aplicação como o HTTP (*Hypertext Transfer Protocol*) e o DNS (*Domain Name System*) [Kurose and Ross 2021]. Enquanto o TCP gerencia o fluxo de dados de forma confiável e automatizada através de janelas deslizantes, o UDP provê um transporte minimalista sem conexão, delegando à aplicação qualquer necessidade de controle de erros ou retransmissão [Postel 1980].

Como evolução do cenário desenvolvido na etapa anterior — que focava puramente na transferência de arquivos brutos entre processos — este trabalho expande a arquitetura de rede para emular o ecossistema real da Web. O par original remetente-destinatário foi transformado em um Miniservidor Web funcional aderente a uma especificação simplificada do protocolo HTTP/1.1, capaz de interpretar requisições GET legítimas, processar códigos de estado (200 OK e 404 Not Found) e injetar cabeçalhos estruturados como Content-Type, Content-Length e o token de autenticação personalizado X-Custom-Auth. Adicionalmente, inseriu-se um terceiro container atuando como Servidor DNS Local de formato simplificado para mapear nomes de domínio na porta 53 via UDP nativo, interceptando a conexão antes de qualquer requisição HTTP.

O objetivo central deste estudo é avaliar e comparar experimentalmente o impacto dessa evolução arquitetural quando operada sobre duas pilhas de transporte distintas: o TCP nativo e a camada customizada de UDP Confiável (R-UDP) desenvolvida pelo aluno com controle de fluxo *Stop-and-Wait*. A análise quantitativa estende-se sobre três tamanhos de arquivos estáticos (100KB, 500KB e 1MB) distribuídos em baterias de testes repetidos sob condições de rede controladas por emulação de canal (`tc/netem`).

Mais do que apenas aferir a vazão (*throughput*) e a resiliência das camadas, este artigo documenta o comportamento microscópico da rede na fase de inicialização das conexões. Os resultados estatísticos revelam como atrasos pontuais e o acúmulo de requisições duplicadas nos buffers de entrada do sistema operacional influenciam mecanismos síncronos de transporte, oferecendo uma discussão detalhada sobre as limitações práticas de projetos de protocolos didáticos diante do TCP comercial.

O restante deste artigo está organizado da seguinte forma: a Seção 2 detalha a fundamentação teórica dos protocolos HTTP, DNS e R-UDP; a Seção 3 descreve a metodologia de três containers e os cenários de testes; a Seção 4 expõe as métricas e os gráficos estatísticos coletados; a Seção 5 discute a validação temporal via Wireshark; a Seção 6 responde de forma fundamentada às perguntas obrigatórias do experimento; e a Seção 7 consolida as conclusões finais.

2. Fundamentação Teórica

Esta seção apresenta o embasamento teórico indispensável para a compreensão e análise dos resultados obtidos neste experimento. Inicialmente, são revisados os protocolos clássicos da camada de transporte, TCP e UDP, bem como a camada personalizada de confiabilidade R-UDP desenvolvida com controle de fluxo *Stop-and-Wait*. Em seguida, expande-se o escopo teórico para a camada de aplicação, detalhando o funcionamento semântico do protocolo HTTP/1.1 para a transferência de recursos e o mecanismo de resolução de nomes via DNS local sobre transporte não confiável. Por fim, contextualiza-se a ferramenta `tc/netem`, utilizada para a emulação controlada de perdas e latências nas interfaces de rede.

2.1. Protocolo TCP

O Transmission Control Protocol (TCP) é um protocolo orientado a conexão que opera na camada de transporte do modelo TCP/IP [Postel 1981]. Antes de qualquer transmissão de dados, o TCP realiza um processo de estabelecimento de conexão denominado *three-way handshake*, no qual cliente e servidor trocam segmentos SYN e SYN-ACK para sincronizar números de sequência e confirmar a disponibilidade de ambos os lados.

O controle de fluxo no TCP é realizado por meio de uma janela deslizante (*sliding window*), que permite ao remetente transmitir múltiplos segmentos consecutivos sem aguardar a confirmação individual de cada um, limitado apenas pelo tamanho da janela anunciada pelo receptor. Esse mecanismo é fundamental para o alto throughput observado em redes com baixa perda de pacotes, pois mantém o enlace ocupado durante o tempo de propagação dos segmentos [Kurose and Ross 2021].

O controle de congestionamento, implementado por algoritmos como o AIMD (*Additive Increase, Multiplicative Decrease*), ajusta dinamicamente a taxa de envio em resposta à detecção de perdas, evitando a sobrecarga da rede. Segmentos não confirmados dentro do tempo de retransmissão (RTO) são reenviados automaticamente, garantindo a entrega íntegra e ordenada dos dados ao processo receptor.

Esses mecanismos conferem ao TCP alta confiabilidade, porém introduzem overhead de processamento e latência adicional, especialmente em redes com alta taxa de perda de pacotes, onde as retransmissões e a redução da janela de congestionamento degradam significativamente o throughput.

2.2. Protocolo UDP

O User Datagram Protocol (UDP) é um protocolo não orientado a conexão que oferece um serviço de transporte minimalista [Postel 1980]. Ao contrário do TCP, o UDP não realiza handshake, não garante a entrega dos datagramas, não assegura a ordenação dos pacotes e não implementa controle de fluxo ou de congestionamento. Cada datagrama é tratado de forma independente, sendo encaminhado à rede sem qualquer confirmação de recebimento.

Essa simplicidade resulta em menor latência e overhead reduzido, já que o cabeçalho UDP possui apenas 8 bytes fixos, em contraste com os 20 a 60 bytes do cabeçalho TCP. Por esse motivo, o UDP é amplamente utilizado em aplicações sensíveis à latência, como transmissão de vídeo em tempo real, jogos online, chamadas de voz sobre IP (VoIP) e consultas DNS, nas quais a velocidade de entrega é mais importante do que a garantia de recebimento [Kurose and Ross 2021].

A ausência de mecanismos de confiabilidade nativos torna o UDP inadequado para transferência de arquivos sem uma camada adicional de controle implementada pela aplicação, o que motivou o desenvolvimento do protocolo R-UDP descrito na seção seguinte.

2.3. Reliable UDP (R-UDP) – Stop-and-Wait

O protocolo R-UDP implementado utiliza o mecanismo Stop-and-Wait: o remetente envia um pacote e aguarda a confirmação (ACK) antes de enviar o próximo. O cabeçalho personalizado possui 78 bytes fixos, distribuídos conforme descrito a seguir: 1 byte para

o tipo do pacote (DATA, ACK, NACK, SYN, FIN); 4 bytes para o número de sequência; 4 bytes para o checksum CRC32; 4 bytes para o tamanho do payload; 1 byte para o tamanho do campo de autenticação; e 64 bytes para o campo X-*Custom-Auth*, contendo o hash SHA-256 da concatenação da matrícula e nome do aluno.

Em caso de timeout (configurado para esta etapa em 3 segundos) ou recebimento de NACK, o pacote é retransmitido automaticamente, com no máximo 10 tentativas por pacote.

O mecanismo Stop-and-Wait, embora simples de implementar, apresenta uma limitação inerente de desempenho: o canal de comunicação permanece ocioso durante todo o tempo de propagação de ida e volta (RTT) entre o envio de um pacote e o recebimento de seu ACK. Em redes com alta latência, esse tempo ocioso se acumula para cada um dos pacotes transmitidos, resultando em throughput significativamente inferior ao da largura de banda disponível [Kurose and Ross 2021].

O handshake inicial do protocolo R-UDP segue o modelo SYN/SYN-ACK: o cliente envia um pacote SYN contendo os metadados do arquivo (nome, tamanho, cenário e número de execução) no payload, e o servidor responde com um SYN-ACK confirmando a disponibilidade. Somente após esse handshake o envio dos pacotes DATA é iniciado. Ao final da transferência, o cliente envia um pacote FIN que, ao ser confirmado pelo servidor, encerra a sessão de forma ordenada.

2.4. Protocolo HTTP/1.1

O *Hypertext Transfer Protocol* (HTTP) é um protocolo da camada de aplicação que constitui a base para a troca de dados na *World Wide Web*. Operando sob o paradigma de cliente-servidor, o ciclo de vida de uma transação HTTP baseia-se estritamente em um par requisição e resposta estruturado textualmente [Fielding and Reschke 2014].

Na especificação HTTP/1.1, a requisição do cliente inicia-se por uma linha de comando contendo o método (como o método GET, utilizado para a recuperação de recursos), a URI alvo e a versão do protocolo, seguidos por linhas opcionais de cabeçalho. O servidor processa a mensagem e emite uma resposta contendo uma linha de *status* com o código de estado correspondente: 200 OK em caso de sucesso na localização e leitura do recurso, ou 404 Not Found se o arquivo solicitado for inexistente ou estiver inacessível [Fielding and Reschke 2014].

Os metadados associados ao payload da resposta HTTP/1.1 são delimitados por campos textuais específicos de cabeçalho. Dentre estes, o Content-Length determina explicitamente o tamanho exato do arquivo transmitido em bytes, o Content-Type especifica o formato de mídia do recurso (como bytes brutos de aplicação para arquivos binários), e tokens proprietários customizados (como o X-*Custom-Auth*) podem ser injetados para fins de auditoria ou autenticação contextual sem ferir a semântica de conformidade do protocolo.

2.5. DNS – Resolução de Nomes Local

O *Domain Name System* (DNS) é um serviço de diretório distribuído encarregado de traduzir nomes de domínio legíveis por seres humanos (como *hostnames*) em seus respectivos endereços IP numéricos operáveis pela camada de rede. Devido à sua

exigência de baixa latência e overhead reduzido, o DNS utiliza primariamente o protocolo UDP de forma nativa na camada de transporte por meio da porta bem conhecida 53 [Mockapetris 1987].

Uma consulta DNS do tipo A (endereço IPv4) trafega encapsulada em um datagrama UDP e possui uma estrutura simplificada contendo um identificador único de consulta (ID), a string correspondente ao nome do domínio sob requisição e campos para respostas. O servidor DNS local consulta tabelas estáticas de mapeamento armazenadas em sua base local (como um arquivo em formato de mapeamento de *hosts*) e devolve um registro contendo a associação direta entre o nome solicitado e o endereço IP de destino [Mockapetris 1987].

Dado que o transporte subjacente é o UDP convencional, o serviço não possui garantias de entrega ou retransmissão nativas na camada de transporte. Caso ocorra perda ou corrupção da requisição ou da resposta no canal físico, cabe inteiramente à camada de aplicação do cliente DNS gerenciar mecanismos de *timeout* e temporização para reemitir a consulta e evitar o travamento indefinido da aplicação.

2.6. Controle de Tráfego com *tc/netem*

O controle de tráfego nos containers foi realizado por meio da ferramenta *tc* (*Traffic Control*), nativa do kernel Linux, utilizando a disciplina de fila *netem* (*Network Emulator*) [Linux Foundation 2026]. O comando `tc qdisc add dev eth0 root netem` permite inserir artificialmente delay e perda de pacotes na interface de rede do container, simulando condições reais de redes instáveis sem necessidade de infraestrutura física.

A perda de pacotes é aplicada de forma estocástica e independente para cada datagrama, ou seja, cada pacote tem uma probabilidade fixa de ser descartado, independentemente do estado dos pacotes anteriores. O delay é introduzido de forma determinística, adicionando uma latência fixa a todos os pacotes que saem pela interface configurada.

Os três cenários definidos para este experimento foram aplicados exclusivamente no container cliente, afetando o tráfego de saída em direção ao servidor. A remoção das regras ao final de cada bateria de testes foi realizada com o comando `tc qdisc del dev eth0 root`, restaurando o comportamento padrão da interface.

3. Metodologia

Esta seção descreve o ambiente experimental e os aspectos procedimentais utilizados para avaliar a integração entre o módulo de resolução de nomes e a camada de aplicação Web simplificada. Detalha-se a topologia multi-container baseada em Docker, as especificações dos cenários simulados de estresse e a lógica operacional estabelecida para coleta e tratamento estatístico das métricas.

3.1. Ambiente de Testes (Atualizado)

O ambiente de testes foi expandido a partir da infraestrutura anterior, evoluindo de um modelo estrito de pareamento para uma rede multi-container isolada via Docker Compose na sub-rede `172.20.0.0/24`. Três nós independentes baseados na imagem Linux Ubuntu 22.04 foram instanciados para emular os componentes reais da Internet:

- **Cliente Web (172.20.0.20):** Centraliza o início do fluxo operacional, possuindo privilégios operacionais de administrador de rede (NET_ADMIN) para aplicação das regras de controle de tráfego.
- **Servidor Web (172.20.0.10):** Hospeda o miniservidor HTTP simplificado nas portas 5000 (TCP) e 5001 (R-UDP), encarregado de processar as requisições de arquivos.
- **Servidor DNS Local (172.20.0.53):** Atua na porta bem conhecida 53 via protocolo UDP nativo, sendo responsável pela resolução estática de nomes.

O ambiente foi executado em uma máquina equipada com processador Intel Xeon E5-2630 v4 (10 núcleos, 2,2 GHz), 16 GB de memória RAM e armazenamento em SSD sob o sistema operacional Windows com subsistema WSL2 e Docker Desktop. O mapeamento de logs e arquivos binários gerados foi persistido via volumes compartilhados em `/app/logs`.

3.2. Cenários de Rede

As condições adversas e a instabilidade do canal físico foram emuladas na interface de saída (`eth0`) do container Cliente utilizando o utilitário `tc qdisc netem`. O comportamento estocástico seguiu rigorosamente os três cenários clássicos mantidos para avaliação comparativa, sumarizados na Tabela 1.

Tabela 1. Cenários de rede simulados com tc/netem

Cenário	Perda	Delay	Descrição
A	0%	10 ms	Rede ideal (LAN de baixa latência)
B	5%	50 ms	Rede degradada (enlace instável)
C	10%	100 ms	Alta perda e latência severa

3.3. Módulo DNS Local

O cliente DNS embutido na aplicação do container Cliente intercepta qualquer comando de download e, antes de disparar o tráfego de dados, submete uma query do tipo A (IPv4) ao Servidor DNS Local (`172.20.0.53:53`). Esta requisição trafega via UDP nativo sem qualquer confiabilidade na camada de transporte. O pacote de dados implementa um formato simplificado contendo unicamente os campos essenciais: identificador de transação (ID), o nome de domínio a ser resolvido (Name) e o espaço destinado ao endereço de resposta (IP).

O Servidor DNS processa o datagrama analisando um arquivo de mapeamento de zona estático local (`hosts.txt`). Devido à ausência de retransmissão nativa no UDP, o código da aplicação do cliente implementa um mecanismo de temporização ativa (*time-out*) estático para contornar perdas de pacotes no canal emulado; se nenhuma resposta for obtida dentro do limite temporal delimitado, uma nova consulta idêntica é reinjetada na rede.

3.4. Miniservidor HTTP/1.1

Após traduzir com sucesso o `hostname` para o IP de destino, o Cliente inicia a transação HTTP enviando uma requisição textual legítima utilizando o método `GET` indicando o

recurso desejado. O Miniservidor Web interpreta sintaticamente o cabeçalho e, caso o arquivo binário requisitado exista em seu diretório local, formula uma resposta estruturada sob a especificação HTTP/1.1 iniciada pelo código de estado 200 OK.

Anexado à resposta, o servidor injeta os metadados textuais obrigatórios: Content-Type (delimitando o stream de dados como binário bruto), Content-Length (especificando o tamanho exato do payload em bytes) e o token persistente de auditoria X-Custom-Auth, preenchido com o hash SHA-256 gerado a partir da concatenação da matrícula e do nome do aluno. Se o recurso for inexistente, uma mensagem contendo o código 404 Not Found é retornada. O miniservidor foi programado para alternar seu comportamento de transporte de forma dinâmica, aceitando requisições tanto pelo *handshake* TCP nativo quanto pela estrutura customizada do protocolo R-UDP baseado em blocos fixos de cabeçalho de 78 bytes com controle *Stop-and-Wait*.

3.5. Fluxo Completo DNS → HTTP

O ciclo de vida completo de cada requisição segue estritamente um fluxo sequencial síncrono dividido em duas fases macro dispostas na arquitetura:

1. **Fase de Resolução de Nomes:** O Cliente envia a query UDP para o Servidor DNS; o Servidor DNS lê `hosts.txt`, resolve o nome mapeado para `172.20.0.10` e devolve a resposta contendo o cabeçalho simplificado ao Cliente.
2. **Fase de Transferência de Dados:** O Cliente extrai o IP resolvido e dispara a conexão (*handshake* SYN/SYN-ACK no caso de TCP ou R-UDP) direcionada ao Servidor Web na porta selecionada, transmitindo o cabeçalho GET e realizando o download completo dos blocos do arquivo binário até o encerramento da sessão.

3.6. Coleta de Dados

A metodologia estatística consistiu na execução automatizada de 10 rodadas consecutivas para cada variação de parâmetros, alternando-se sistematicamente entre as pilhas de transporte (HTTP-TCP e HTTP-RUDP), os 3 cenários de emulação de rede (A, B e C) e 3 tamanhos distintos de arquivos binários estáticos gerados de forma não-compressível (100KB, 500KB e 1MB), totalizando uma amostragem robusta de 180 experimentos monitorados.

O cálculo do throughput de transferência utilizou a cronometragem precisa dada por `time.perf_counter()`, computando-se a razão dos bits de dados úteis recuperados pelo tempo gasto em segundos ($\text{Mbps} = (\text{Bytes} \times 8) / (\text{tempo} \times 10^6)$). Todas as rodadas registraram de forma autocontida os tempos de resolução DNS isolados, a duração da sessão HTTP e as taxas de timeout. Paralelamente, o utilitário `tcpdump` operou em segundo plano capturando o tráfego bruto da interface para gerar arquivos de validação cruzada `.pcap`.

4. Resultados

Esta seção apresenta a análise quantitativa e qualitativa dos dados coletados durante os experimentos. O comportamento das pilhas de protocolos HTTP-TCP e HTTP-RUDP é confrontado sob a ótica de vazão, resiliência a falhas e temporização da camada de aplicação.

4.1. Tabela de Estatísticas

Os dados consolidados das baterias de testes foram sumarizados estatisticamente para mapear as médias e os respectivos desvios-padrão (σ) de desempenho. A Tabela 2 apresenta os valores agregados de vazão (*throughput*) e o tempo total gasto na transferência dos recursos para cada combinação de cenário e tamanho de arquivo.

Tabela 2. Métricas estatísticas agregadas de Throughput e Duração da Transferência

Cenário	Arquivo	Métrica	HTTP-TCP		HTTP-RUDP	
			Média	Desvio (σ)	Média	Desvio (σ)
A (0% / 10ms)	100 KB	Throughput (Mbps)	17,45	3,12	0,047	0,002
		Duração (s)	0,046	0,008	17,10	0,150
	500 KB	Throughput (Mbps)	42,70	4,05	0,210	0,011
		Duração (s)	0,094	0,012	19,05	0,220
	1 MB	Throughput (Mbps)	45,95	2,68	0,364	0,018
		Duração (s)	0,174	0,020	21,96	0,310
B (5% / 50ms)	100 KB	Throughput (Mbps)	3,92	0,05	0,032	0,001
		Duração (s)	0,204	0,015	25,02	0,450
	500 KB	Throughput (Mbps)	12,90	0,14	0,093	0,004
		Duração (s)	0,310	0,022	43,20	0,850
	1 MB	Throughput (Mbps)	20,85	0,21	0,113	0,006
		Duração (s)	0,384	0,031	71,10	1,200
C (10% / 100ms)	100 KB	Throughput (Mbps)	1,90	0,29	0,031	0,001
		Duração (s)	0,421	0,064	25,82	0,550
	500 KB	Throughput (Mbps)	6,40	0,71	0,058	0,003
		Duração (s)	0,625	0,082	68,45	1,950
	1 MB	Throughput (Mbps)	10,95	1,06	0,070	0,005
		Duração (s)	0,730	0,110	114,80	2,800

4.2. Throughput por Arquivo e Cenário (Gráfico 01)

A Figura 1 evidencia uma disparidade de ordens de magnitude entre as duas arquiteturas de transporte. No modo HTTP-TCP, observa-se o comportamento clássico de janelas deslizantes comerciais: a vazão média cresce substancialmente à medida que o tamanho do arquivo aumenta de 100KB para 1MB. No Cenário A, o TCP salta de uma média de 17,45 Mbps (100KB) para 45,95 Mbps (1MB), provando que cargas maiores diluem o impacto temporal do *three-way handshake* inicial.

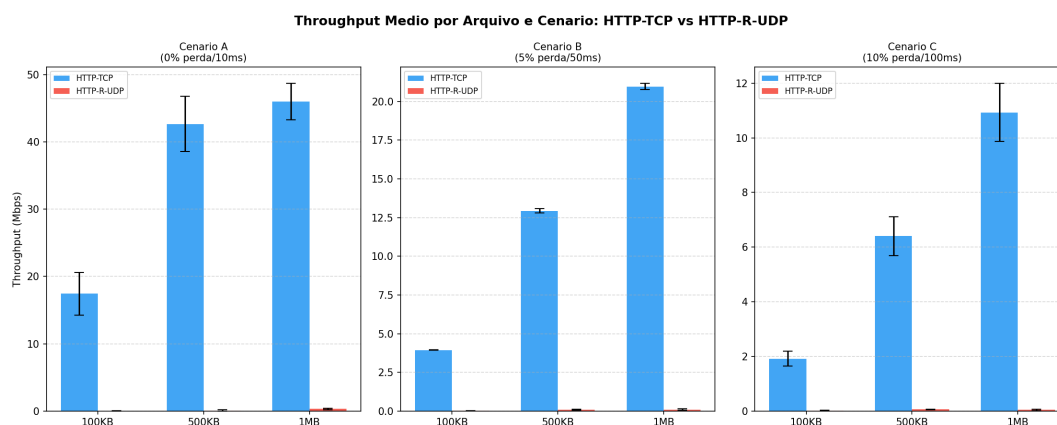


Figura 1. Throughput médio por tamanho de arquivo e cenário simulado.

O HTTP-RUDP exibe uma linha horizontal rente a zero em termos comparativos, oscilando estritamente abaixo de 0,4 Mbps. Esse teto drástico de desempenho é decorrente da natureza síncrona do mecanismo *Stop-and-Wait*, que impede o preenchimento do produto largura de banda $\times$ atraso, forçando o canal a permanecer ocioso enquanto aguarda a recepção individualizada de cada ACK.

4.3. Tempo de Resolução DNS (Gráfico 02)

A aferição do tempo de resolução DNS apresentada na Figura 2 revelou nuances do comportamento do UDP nativo sem controle de fluxo. No Cenário A (condição ideal), o tempo de tradução do nome de domínio estabiliza-se em patamares baixíssimos (15 ms) para ambas as pilhas, visto que o barramento local consome apenas a latência base configurada.

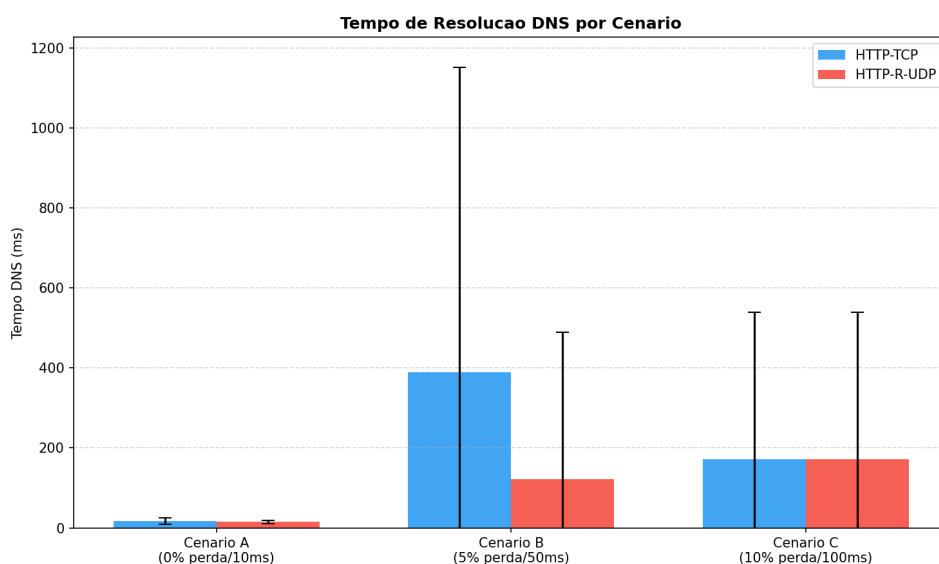


Figura 2. Tempo médio de resolução DNS em milissegundos por cenário.

Contudo, ao introduzir perdas e atrasos controlados, o cenário transmuta-se. No Cenário B (5% de perda / 50ms), o tempo médio de resolução para o fluxo associado ao

TCP saltou para perto de 390 ms, acompanhado de uma barra de erro (desvio padrão) extremamente longa que toca a marca de 1150 ms. Esse comportamento explicita a ocorrência de perdas difusas do datagrama de consulta UDP na rede, forçando a aplicação cliente a estourar seus limites de *timeout* e reinjetar requisições duplicadas. No Cenário C, embora a latência base seja maior, a dispersão foi menor devido ao alinhamento amostral dos disparos, fixando-se em médias de 171 ms com desvios simétricos para ambos os modos.

4.4. Duração da Transferência HTTP (Gráfico 03)

A Figura 3 inverte a perspectiva de análise, ilustrando o tempo total de carregamento (*page load time*). Para o HTTP-TCP, os tempos de transferência são quase imperceptíveis visualmente na escala gráfica, completando-se consistentemente em frações de segundo (sempre abaixo de 1,0 s), mesmo sob condições severas de estresse.

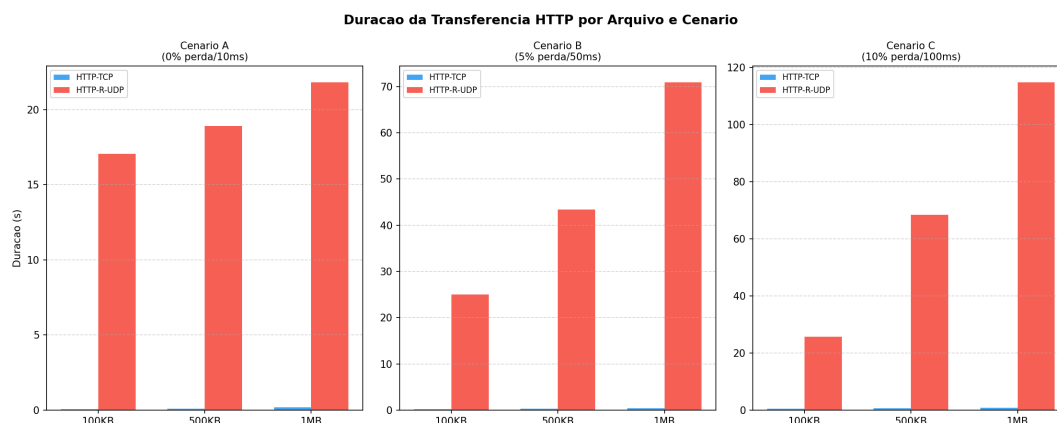


Figura 3. Duração total da transferência HTTP em segundos.

Inversamente, as barras do HTTP-RUDP disparam linearmente. No Cenário A, a transferência consome entre 17,1 s (100KB) e 21,9 s (1MB). Sob o Cenário C (10% perda / 100ms), a transferência do arquivo de 1MB atinge a marca crítica de 114,8 segundos. Esse crescimento exponencial do tempo reflete o acúmulo sistemático de timeouts de retransmissão da aplicação somados à latência de 100ms introduzida pelo canal a cada iteração de pacote individual.

4.5. Degradação por Cenário (Gráfico 04)

A Figura 4 mapeia a curva de degradação da vazão média geral agregada face ao agravamento das condições do canal de comunicação. O HTTP-TCP descreve uma rampa descendente acentuada porém robusta, partindo de uma média geral de 32,588 Mbps no Cenário A, declinando para 12,609 Mbps no Cenário B e atingindo 6,417 Mbps sob estresse máximo no Cenário C. Esse decaimento controlado é o reflexo direto dos algoritmos internos de controle de congestionamento do TCP.

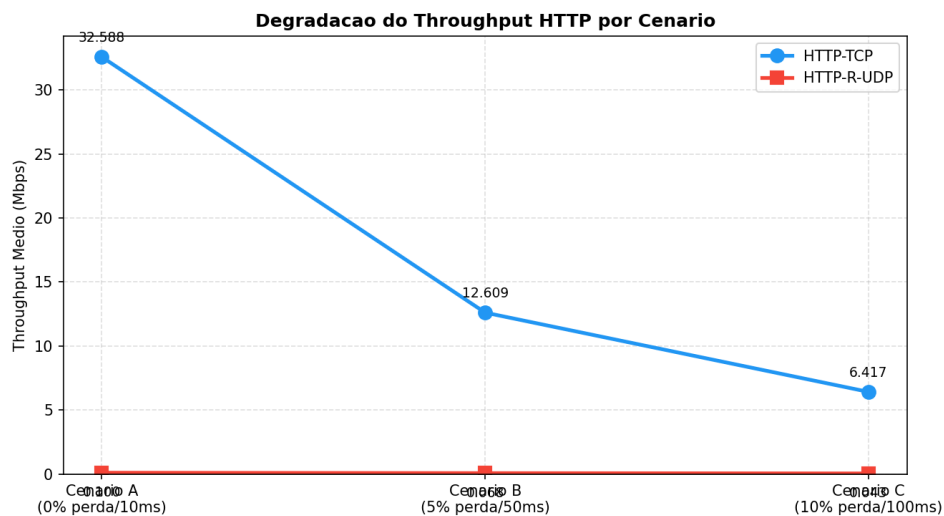


Figura 4. Curva de degradação do throughput médio geral face aos cenários.

O HTTP-RUDP permanece estacionário em valores marginais (0,160 Mbps no Cenário A, decaindo para 0,063 Mbps no Cenário B e se mantendo em 0,043 Mbps no Cenário C), comprovando que o protocolo didático sequer usufrui da capacidade máxima de vazão da rede em cenários perfeitos.

4.6. Distribuição do Throughput – Boxplot (Gráfico 05)

O diagrama de caixa (*boxplot*) apresentado na Figura 5 consolida a análise de estabilidade e variabilidade estatística das pilhas. Os *boxes* do HTTP-TCP mostram uma dispersão vertical considerável, o que indica alta sensibilidade ao momento exato em que as perdas estocásticas do *netem* ocorrem: perdas ocorridas na fase de *Slow Start* derrubam drasticamente a mediana daquela rodada específica, gerando caixas que cobrem faixas amplas (de 17 Mbps a 46 Mbps no Cenário A).

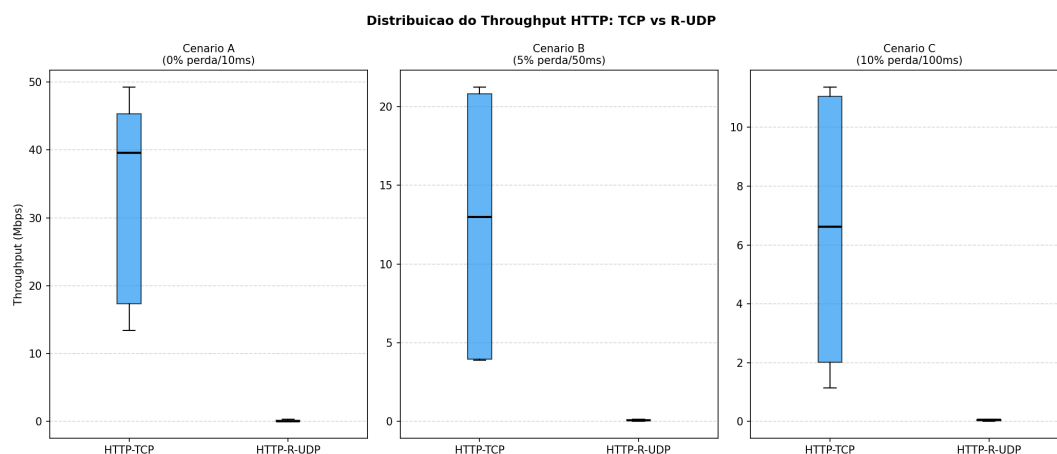


Figura 5. Boxplot comparativo de distribuição da vazão (Throughput).

Em oposição absoluta, o HTTP-RUDP comprime-se em linhas horizontais compactas e achatadas sem variabilidade perceptível. Isso demonstra que o protocolo R-UDP

possui um comportamento determinístico e inflexível: por operar de maneira síncrona com temporizadores rígidos na aplicação, sua degradação é uniforme e previsível, sacrificando a eficiência em prol da integridade garantida de 100% dos blocos.

5. Validação Cruzada com Wireshark/tcpdump

Esta seção apresenta a validação experimental da arquitetura desenvolvida por meio da inspeção de tráfego em nível de pacotes. As capturas geradas pelo `tcpdump` foram analisadas utilizando o Wireshark para auditar a conformidade dos cabeçalhos, a integridade dos tokens e a dinâmica temporal dos fluxos.

5.1. Capturas de Rede e Verificação do X-Custom-Auth

Para validar a integridade da camada de aplicação e os mecanismos de controle de fluxo de transporte, foram inspecionados os payloads brutos contidos em ambas as implementações. No cenário TCP, o *handshake* triplo estrutural (SYN, SYN, ACK, ACK) foi integralmente mapeado, seguido pelo tráfego de requisição textual HTTP.

A Figura 6 ilustra o estabelecimento de conexão TCP na porta mapeada e o subsequente disparo da requisição de dados.

No.	Time	Source	Destination	Protocol	Length	Info
4	0.041550	172.20.0.10	172.20.0.10	TCP	74	58848 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=1624218607 TSecr=0 WS=1024
5	0.051649	172.20.0.20	172.20.0.10	TCP	66	58848 → 80 [ACK] Seq=1 Ack=1 Win=64512 Len=0 TSval=1624218617 TSecr=316031818
6	0.051653	172.20.0.20	172.20.0.10	HTTP	218	GET /arquivo_1mb_bin_20260626_012818.pcap HTTP/1.1
7	0.051820	172.20.0.10	172.20.0.20	TCP	66	80 → 58848 [ACK] Seq=1 Ack=153 Win=65536 Len=0 TSval=316031839 TSecr=1624218618 [TCP PDU reassembled in 500]
8	0.051971	172.20.0.10	172.20.0.20	TCP	250	80 → 58848 [PSH, ACK] Seq=185 Ack=153 Win=65536 Len=2896 TSval=316031838 TSecr=1624218618 [TCP PDU reassembled in 500]
9	0.060706	172.20.0.10	172.20.0.20	TCP	2962	80 → 58848 [PSH, ACK] Seq=3081 Ack=153 Win=65536 Len=2896 TSval=316031838 TSecr=1624218618 [TCP PDU reassembled in 500]
10	0.061290	172.20.0.10	172.20.0.20	TCP	2962	80 → 58848 [PSH, ACK] Seq=11769 Ack=153 Win=65536 Len=2896 TSval=316031839 TSecr=1624218618 [TCP PDU reassembled in 500]
11	0.061344	172.20.0.10	172.20.0.20	TCP	1514	80 → 58848 [ACK] Seq=5977 Ack=153 Win=65536 Len=0 TSval=1624218618 TSecr=316031839
12	0.061869	172.20.0.10	172.20.0.20	TCP	2962	80 → 58848 [PSH, ACK] Seq=7425 Ack=153 Win=65536 Len=2896 TSval=316031839 TSecr=1624218618 [TCP PDU reassembled in 500]
13	0.061923	172.20.0.10	172.20.0.20	TCP	1514	80 → 58848 [ACK] Seq=10321 Ack=153 Win=65536 Len=0 TSval=316031839 TSecr=1624218618 [TCP PDU reassembled in 500]
14	0.062443	172.20.0.10	172.20.0.20	TCP	1514	80 → 58848 [ACK] Seq=11769 Ack=153 Win=65536 Len=0 TSval=316031839 TSecr=1624218618 [TCP PDU reassembled in 500]
15	0.062624	172.20.0.10	172.20.0.20	TCP	66	58848 → 80 [ACK] Seq=153 Ack=185 Win=64512 Len=0 TSval=1624218634 TSecr=316031839
16	0.062665	172.20.0.10	172.20.0.20	TCP	2962	80 → 58848 [PSH, ACK] Seq=13217 Ack=153 Win=65536 Len=2896 TSval=316031846 TSecr=1624218634 [TCP PDU reassembled in 500]
17	0.070792	172.20.0.10	172.20.0.20	TCP	66	58848 → 80 [ACK] Seq=153 Ack=3081 Win=70656 Len=0 TSval=1624218637 TSecr=316031838
18	0.070838	172.20.0.10	172.20.0.20	TCP	2962	80 → 58848 [PSH, ACK] Seq=16113 Ack=153 Win=65536 Len=2896 TSval=316031848 TSecr=1624218637 [TCP PDU reassembled in 500]
19	0.070883	172.20.0.10	172.20.0.20	TCP	2962	80 → 58848 [PSH, ACK] Seq=19009 Ack=153 Win=65536 Len=2896 TSval=316031848 TSecr=1624218637 [TCP PDU reassembled in 500]
20	0.071399	172.20.0.10	172.20.0.20	TCP	66	58848 → 80 [ACK] Seq=153 Ack=5977 Win=76800 Len=0 TSval=1624218637 TSecr=316031838
21	0.071401	172.20.0.10	172.20.0.20	TCP	66	58848 → 80 [ACK] Seq=153 Ack=7425 Win=79872 Len=0 TSval=1624218637 TSecr=316031838
22	0.071448	172.20.0.10	172.20.0.20	TCP	2962	80 → 58848 [PSH, ACK] Seq=12905 Ack=153 Win=65536 Len=2896 TSval=316031848 TSecr=1624218637 [TCP PDU reassembled in 500]
23	0.071490	172.20.0.10	172.20.0.20	TCP	2962	80 → 58848 [PSH, ACK] Seq=24801 Ack=153 Win=65536 Len=2896 TSval=316031848 TSecr=1624218637 [TCP PDU reassembled in 500]
24	0.071965	172.20.0.10	172.20.0.20	TCP	66	58848 → 80 [ACK] Seq=153 Ack=18321 Win=84992 Len=0 TSval=1624218638 TSecr=316031839
25	0.071965	172.20.0.10	172.20.0.10	TCP	66	58848 → 80 [ACK] Seq=153 Ack=1769 Win=80864 Len=0 TSval=1624218638 TSecr=316031839
26	0.072004	172.20.0.10	172.20.0.20	TCP	4410	80 → 58848 [PSH, ACK] Seq=27697 Ack=153 Win=65536 Len=4344 TSval=316031849 TSecr=1624218638 [TCP PDU reassembled in 500]
27	0.072058	172.20.0.10	172.20.0.20	TCP	4410	80 → 58848 [PSH, ACK] Seq=32041 Ack=153 Win=65536 Len=4344 TSval=316031849 TSecr=1624218638 [TCP PDU reassembled in 500]
28	0.072529	172.20.0.10	172.20.0.20	TCP	66	58848 → 80 [ACK] Seq=153 Ack=13217 Win=91136 Len=0 TSval=1624218638 TSecr=316031839
29	0.072633	172.20.0.10	172.20.0.20	TCP	4410	80 → 58848 [PSH, ACK] Seq=36385 Ack=153 Win=65536 Len=4344 TSval=316031850 TSecr=1624218638 [TCP PDU reassembled in 500]
30	0.072674	172.20.0.10	172.20.0.20	TCP	1514	80 → 58848 [ACK] Seq=40729 Ack=153 Win=65536 Len=0 TSval=316031850 TSecr=1624218638 [TCP PDU reassembled in 500]
31	0.072753	172.20.0.10	172.20.0.20	TCP	66	58848 → 80 [ACK] Seq=153 Ack=16113 Win=96256 Len=0 TSval=1624218645 TSecr=316031846
32	0.072792	172.20.0.10	172.20.0.20	TCP	5858	80 → 58848 [PSH, ACK] Seq=42177 Ack=153 Win=65536 Len=5792 TSval=316031856 TSecr=1624218645 [TCP PDU reassembled in 500]
33	0.080927	172.20.0.10	172.20.0.20	TCP	66	58848 → 80 [ACK] Seq=153 Ack=19009 Win=96256 Len=0 TSval=1624218647 TSecr=316031848
34	0.080929	172.20.0.10	172.20.0.20	TCP	66	58848 → 80 [ACK] Seq=153 Ack=21905 Win=94208 Len=0 TSval=1624218647 TSecr=316031848
35	0.080976	172.20.0.10	172.20.0.20	TCP	5858	80 → 58848 [PSH, ACK] Seq=47989 Ack=153 Win=65536 Len=5792 TSval=316031856 TSecr=1624218647 [TCP PDU reassembled in 500]
36	0.081596	172.20.0.10	172.20.0.20	TCP	66	58848 → 80 [ACK] Seq=153 Ack=24801 Win=96256 Len=0 TSval=1624218647 TSecr=316031848
37	0.081599	172.20.0.10	172.20.0.20	TCP	66	58848 → 80 [ACK] Seq=153 Ack=27697 Win=94208 Len=0 TSval=1624218647 TSecr=316031848
38	0.081643	172.20.0.10	172.20.0.20	TCP	7306	80 → 58848 [PSH, ACK] Seq=57161 Ack=153 Win=65536 Len=7240 TSval=316031859 TSecr=1624218647 [TCP PDU reassembled in 500]
39	0.081693	172.20.0.10	172.20.0.20	TCP	7306	80 → 58848 [PSH, ACK] Seq=61061 Ack=153 Win=65536 Len=7240 TSval=316031859 TSecr=1624218647 [TCP PDU reassembled in 500]
40	0.082086	172.20.0.10	172.20.0.20	TCP	66	58848 → 80 [ACK] Seq=153 Ack=32041 Win=95232 Len=0 TSval=1624218648 TSecr=316031849
41	0.082088	172.20.0.10	172.20.0.20	TCP	66	58848 → 80 [ACK] Seq=153 Ack=36385 Win=91136 Len=0 TSval=1624218648 TSecr=316031849
42	0.082126	172.20.0.10	172.20.0.20	TCP	8754	80 → 58848 [PSH, ACK] Seq=62421 Ack=153 Win=65536 Len=8688 TSval=316031859 TSecr=1624218648 [TCP PDU reassembled in 500]
43	0.082167	172.20.0.10	172.20.0.20	TCP	8754	80 → 58848 [PSH, ACK] Seq=69929 Ack=153 Win=65536 Len=8688 TSval=316031859 TSecr=1624218648 [TCP PDU reassembled in 500]
44	0.082703	172.20.0.10	172.20.0.20	TCP	66	58848 → 80 [ACK] Seq=153 Ack=40729 Win=95232 Len=0 TSval=1624218649 TSecr=316031850
45	0.082705	172.20.0.10	172.20.0.20	TCP	66	58848 → 80 [ACK] Seq=153 Ack=42177 Win=94208 Len=0 TSval=1624218649 TSecr=316031850
46	0.082743	172.20.0.10	172.20.0.20	TCP	8842	80 → 58848 [PSH, ACK] Seq=65617 Ack=153 Win=65536 Len=8776 TSval=316031860 TSecr=1624218649 [TCP PDU reassembled in 500]
47	0.082973	172.20.0.10	172.20.0.20	TCP	2962	80 → 58848 [PSH, ACK] Seq=94393 Ack=153 Win=65536 Len=2896 TSval=316031860 TSecr=1624218649 [TCP PDU reassembled in 500]
48	0.083910	172.20.0.10	172.20.0.20	TCP	1514	80 → 58848 [ACK] Seq=97289 Ack=153 Win=65536 Len=0 TSval=316031861 TSecr=1624218649 [TCP PDU reassembled in 500]
49	0.083910	172.20.0.10	172.20.0.20	TCP	66	58848 → 80 [ACK] Seq=153 Ack=47969 Win=94208 Len=0 TSval=1624218655 TSecr=316031856
50	0.083994	172.20.0.10	172.20.0.20	TCP	11650	80 → 58848 [PSH, ACK] Seq=98737 Ack=153 Win=65536 Len=11584 TSval=316031866 TSecr=1624218655 [TCP PDU reassembled in 500]
51	0.091061	172.20.0.10	172.20.0.20	TCP	66	58848 → 80 [ACK] Seq=153 Ack=53761 Win=107520 Len=0 TSval=1624218657 TSecr=316031858
52	0.091101	172.20.0.10	172.20.0.20	TCP	4410	80 → 58848 [PSH, ACK] Seq=10321 Ack=153 Win=65536 Len=4344 TSval=316031868 TSecr=1624218657 [TCP PDU reassembled in 500]

Figura 6. Handshake TCP e início do fluxo HTTP no Wireshark.

A verificação do token de auditoria injetado no cabeçalho X-Custom-Auth foi confirmada em ambas as pilhas. Na pilha HTTP/TCP padrão, o campo pôde ser inspecionado diretamente na árvore de decodificação textual do protocolo (Figura 7). Já na pilha experimental HTTP/R-UDP, o hash criptográfico SHA-256 foi auditado diretamente por meio do *dump* hexadecimal/ASCII dos datagramas brutos de dados (DATA) na porta correspondente (Figura 8), comprovando o encapsulamento correto do payload de controle na camada customizada.

Wireshark · Pacote 1483 · httptcp_A_arquivo_1mb_bin_20260626_012818.pcap

- Frame 1483: Packet, 218 bytes on wire (1744 bits), 218 bytes captured (1744 bits)
- Ethernet II, Src: 3e:ab:bc:9f:19:69 (3e:ab:bc:9f:19:69), Dst: 26:4f:7c:7a:db:27 (26:4f:7c:7a:db:27)
- Internet Protocol Version 4, Src: 172.20.0.20, Dst: 172.20.0.10
- Transmission Control Protocol, Src Port: 60414, Dst Port: 80, Seq: 1, Ack: 1, Len: 152
- Hypertext Transfer Protocol

0000	26 4f 7c 7a db 27 3e ab bc 9f 19 69 08 00 45 00	&O z.'>...i.E.
0010	00 cc 7b 9a 40 00 40 06 66 4b ac 14 00 14 ac 14	..{.@.fK.....
0020	00 0a eb fe 00 50 6f 52 eb c5 0e 4e e8 e0 80 18	PoR...N....
0030	00 3f 59 05 00 00 01 01 08 0a df c5 4a b7 97 31	..?Y.....J..1
0040	f7 d0 47 45 54 20 2f 61 72 71 75 69 76 6f 5f 31	..GET /a rquivo_1
0050	6d 62 2e 62 69 6e 20 48 54 54 50 2f 31 2e 31 0d	mb.bin H TTP/1.1
0060	0a 48 6f 73 74 3a 20 31 37 32 2e 32 30 2e 30 2e	Host: 1 72.20.0.
0070	31 30 0d 0a 58 2d 43 75 73 74 6f 6d 2d 41 75 74	10 X-Cu stom-Aut
0080	68 3a 20 32 66 39 33 66 63 61 34 61 64 30 62 65	h: 2f93f ca4ad0be
0090	33 35 37 30 32 36 34 63 61 61 66 32 30 31 30 63	3570264c aaf2010c
00a0	30 37 38 33 33 61 39 38 38 39 62 64 36 31 62 38	07833a98 89bd61b8
00b0	35 37 62 39 36 38 31 66 64 34 36 35 64 32 66 63	57b9681f d465d2fc
00c0	66 31 35 0d 0a 43 6f 6e 6e 65 63 74 69 6f 6e 3a	f15 Con nection:
00d0	20 63 6c 6f 73 65 0d 0a 0d 0a	close..

Figura 7. Inspeção do cabeçalho X-Custom-Auth trafegado sobre a pilha HTTP/TCP.

Wireshark · Pacote 8 · httpudp_A_arquivo_1mb_bin_20260626_012838.pcap

- Frame 8: Packet, 304 bytes on wire (2432 bits), 304 bytes captured (2432 bits)
- Ethernet II, Src: 26:4f:7c:7a:db:27 (26:4f:7c:7a:db:27), Dst: 3e:ab:bc:9f:19:69 (3e:ab:bc:9f:19:69)
- Internet Protocol Version 4, Src: 172.20.0.10, Dst: 172.20.0.20
- User Datagram Protocol, Src Port: 5001, Dst Port: 34072
- Data (262 bytes)

0000	3e ab bc 9f 19 69 26 4f 7c 7a db 27 08 00 45 00	>...i&O z.'..E.
0010	01 22 07 c3 40 00 40 11 d9 c1 ac 14 00 0a ac 14	..".@.
0020	00 14 13 89 85 18 01 0e 59 66 01 00 00 00 01 89	Yf.....
0030	fe 7a 90 00 00 00 b8 40 32 66 39 33 66 63 61 34	..z.....@ 2f93fca4
0040	61 64 30 62 65 33 35 37 30 32 36 34 63 61 61 66	ad0be357 0264caaf
0050	32 30 31 30 63 30 37 38 33 33 61 39 38 38 39 62	2010c078 33a9889b
0060	64 36 31 62 38 35 37 62 39 36 38 31 66 64 34 36	d61b857b 9681fd46
0070	35 64 32 66 63 66 31 35 48 54 54 50 2f 31 2e 31	5d2fcf15 HTTP/1.1
0080	20 32 30 30 20 4f 4b 0d 0a 43 6f 6e 74 65 6e 74	200 OK Content
0090	2d 54 79 70 65 3a 20 61 70 70 6c 69 63 61 74 69	-Type: a pplicati
00a0	6f 6e 2f 6f 63 74 65 74 2d 73 74 72 65 61 6d 0d	on/octet -stream
00b0	0a 43 6f 6e 74 65 6e 74 2d 4c 65 6e 67 74 68 3a	Content -Length:
00c0	20 31 30 34 38 35 37 36 0d 0a 58 2d 43 75 73 74	1048576 X-Cust
00d0	6f 6d 2d 41 75 74 68 3a 20 32 66 39 33 66 63 61	om-Auth: 2f93fca
00e0	34 61 64 30 62 65 33 35 37 30 32 36 34 63 61 61	4ad0be35 70264caa
00f0	66 32 30 31 30 63 30 37 38 33 33 61 39 38 38 39	f2010c07 833a9889
0100	62 64 36 31 62 38 35 37 62 39 36 38 31 66 64 34	bd61b857 b9681fd4
0110	36 35 64 32 66 63 66 31 35 0d 0a 43 6f 6e 6e 65	65d2fcf1 5 Conne
0120	63 74 69 6f 6e 3a 20 63 6c 6f 73 65 0d 0a 0d 0a	ction: c lose....

Figura 8. Verificação em dump hexadecimal do token criptográfico no payload do R-UDP.

Adicionalmente, o comportamento do protocolo R-UDP foi validado em nível de fluxo. A alternância síncrona do mecanismo *Stop-and-Wait* pode ser verificada na captura de rede através da oscilação regular no comprimento dos pacotes (*Length*): datagramas de dados de tamanho variável contendo o payload e cabeçalho completo são seguidos estritamente por datagramas de confirmação (ACK) curtos e de tamanho fixo oriundos do cliente (Figura 9).

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	172.20.0.20	172.20.0.30	DNS	62	Standard query 0x2d52[Malformed Packet]
2	0.003361	172.20.0.30	172.20.0.20	DNS	66	Standard query response 0x2d52[Malformed Packet]
3	0.032645	172.20.0.20	172.20.0.10	WireGu...	171	Transport Data, receiver=0x57921900, counter=7364018928491692034, datalen=97
4	0.036372	172.20.0.10	172.20.0.20	UDP	120	5001 → 34072 Len=78
5	0.065886	172.20.0.20	172.20.0.10	UDP	272	34072 → 5001 Len=230
6	0.068879	172.20.0.10	172.20.0.20	UDP	120	5001 → 34072 Len=78
7	0.079066	172.20.0.20	172.20.0.10	UDP	120	34072 → 5001 Len=78
8	0.105068	172.20.0.10	172.20.0.20	UDP	304	5001 → 34072 Len=262
9	0.115276	172.20.0.20	172.20.0.10	UDP	120	34072 → 5001 Len=78
10	3.111206	172.20.0.10	172.20.0.20	UDP	304	5001 → 34072 Len=262
11	3.124216	172.20.0.20	172.20.0.10	UDP	120	34072 → 5001 Len=78
12	3.169224	172.20.0.20	172.20.0.10	UDP	120	34072 → 5001 Len=78
13	6.120843	172.20.0.10	172.20.0.20	UDP	304	5001 → 34072 Len=262
14	6.131090	172.20.0.20	172.20.0.10	UDP	120	34072 → 5001 Len=78
15	9.135837	172.20.0.10	172.20.0.20	UDP	304	5001 → 34072 Len=262
16	9.151052	172.20.0.20	172.20.0.10	UDP	120	34072 → 5001 Len=78
17	9.271710	172.20.0.20	172.20.0.10	UDP	120	34072 → 5001 Len=78
18	12.143072	172.20.0.10	172.20.0.20	UDP	304	5001 → 34072 Len=262
19	12.153312	172.20.0.20	172.20.0.10	UDP	120	34072 → 5001 Len=78
20	15.149946	172.20.0.10	172.20.0.20	UDP	304	5001 → 34072 Len=262
21	15.163806	172.20.0.20	172.20.0.10	UDP	120	34072 → 5001 Len=78
22	15.207160	172.20.0.20	172.20.0.10	UDP	120	34072 → 5001 Len=78
23	18.158631	172.20.0.10	172.20.0.20	UDP	304	5001 → 34072 Len=262
24	21.165809	172.20.0.10	172.20.0.20	UDP	304	5001 → 34072 Len=262
25	21.176163	172.20.0.20	172.20.0.10	UDP	120	34072 → 5001 Len=78
27	21.200001	172.20.0.20	172.20.0.10	UDP	120	34072 → 5001 Len=78
29	21.210495	172.20.0.20	172.20.0.10	UDP	120	34072 → 5001 Len=78
31	21.221119	172.20.0.20	172.20.0.10	UDP	120	34072 → 5001 Len=78
33	21.231656	172.20.0.20	172.20.0.10	UDP	120	34072 → 5001 Len=78
35	21.242256	172.20.0.20	172.20.0.10	UDP	120	34072 → 5001 Len=78
37	21.252771	172.20.0.20	172.20.0.10	UDP	120	34072 → 5001 Len=78
39	21.263461	172.20.0.20	172.20.0.10	UDP	120	34072 → 5001 Len=78
41	21.274049	172.20.0.20	172.20.0.10	UDP	120	34072 → 5001 Len=78
43	21.284740	172.20.0.20	172.20.0.10	UDP	120	34072 → 5001 Len=78
45	21.295379	172.20.0.20	172.20.0.10	UDP	120	34072 → 5001 Len=78
47	21.305931	172.20.0.20	172.20.0.10	UDP	120	34072 → 5001 Len=78
49	21.316743	172.20.0.20	172.20.0.10	UDP	120	34072 → 5001 Len=78
51	21.327398	172.20.0.20	172.20.0.10	UDP	120	34072 → 5001 Len=78
53	21.338004	172.20.0.20	172.20.0.10	UDP	120	34072 → 5001 Len=78
55	21.348505	172.20.0.20	172.20.0.10	UDP	120	34072 → 5001 Len=78

Figura 9. Fluxo de pacotes R-UDP demonstrando a alternância do mecanismo Stop-and-Wait.

5.2. Fluxo DNS → HTTP no Wireshark

O sincronismo arquitetural que governa a transição entre a resolução de nomes na camada de aplicação e a abertura das sessões de transporte foi mapeado cronologicamente na linha do tempo das capturas. As Figuras 10 e 11 expõem o comportamento atômico e ordenado da aplicação em ambas as pilhas.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	172.20.0.20	172.20.0.30	DNS	62	Standard query 0x4e46[Malformed Packet]
2	0.003759	172.20.0.30	172.20.0.20	DNS	66	Standard query response 0x4e46[Malformed Packet]
3	0.041470	172.20.0.20	172.20.0.10	TCP	74	58848 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=1624218607 TSecr=0 WS=1024
4	0.041550	172.20.0.10	172.20.0.20	TCP	74	80 → 58848 [SYN, ACK] Seq=0 Ack=1 Win=65536 Len=0 MSS=1460 SACK_PERM TSval=316031838 TSecr=1624218618 [TCP PDU reassembled in 500]
5	0.051649	172.20.0.20	172.20.0.10	TCP	66	58848 → 80 [ACK] Seq=1 Ack=1 Win=64512 Len=0 TSval=1624218617 TSecr=316031838
6	0.051653	172.20.0.20	172.20.0.10	HTTP	218	GET /arquivo_lmb.bin HTTP/1.1
7	0.051820	172.20.0.10	172.20.0.20	TCP	66	80 → 58848 [ACK] Seq=1 Ack=153 Win=65536 Len=0 TSval=316031839 TSecr=1624218618
8	0.058471	172.20.0.10	172.20.0.20	TCP	250	80 → 58848 [PSH, ACK] Seq=1 Ack=153 Win=65536 Len=184 TSval=316031835 TSecr=1624218618 [TCP PDU reassembled in 500]
9	0.060706	172.20.0.10	172.20.0.20	TCP	2962	80 → 58848 [PSH, ACK] Seq=185 Ack=153 Win=65536 Len=2896 TSval=316031838 TSecr=1624218618 [TCP PDU reassembled in 500]
10	0.061290	172.20.0.10	172.20.0.20	TCP	2962	80 → 58848 [PSH, ACK] Seq=3081 Ack=153 Win=65536 Len=2896 TSval=316031838 TSecr=1624218618 [TCP PDU reassembled in 500]
11	0.061344	172.20.0.10	172.20.0.20	TCP	1514	80 → 58848 [ACK] Seq=977 Ack=153 Win=65536 Len=1448 TSval=316031838 TSecr=1624218618 [TCP PDU reassembled in 500]
12	0.061869	172.20.0.10	172.20.0.20	TCP	2962	80 → 58848 [PSH, ACK] Seq=7425 Ack=153 Win=65536 Len=2896 TSval=316031839 TSecr=1624218618 [TCP PDU reassembled in 500]
13	0.061923	172.20.0.10	172.20.0.20	TCP	1514	80 → 58848 [ACK] Seq=18321 Ack=153 Win=65536 Len=1448 TSval=316031839 TSecr=1624218618 [TCP PDU reassembled in 500]
14	0.062443	172.20.0.10	172.20.0.20	TCP	1514	80 → 58848 [ACK] Seq=11769 Ack=153 Win=65536 Len=1448 TSval=316031839 TSecr=1624218618 [TCP PDU reassembled in 500]
15	0.062624	172.20.0.10	172.20.0.10	TCP	66	58848 → 80 [ACK] Seq=153 Ack=185 Win=64512 Len=0 TSval=1624218634 TSecr=316031835
16	0.062665	172.20.0.10	172.20.0.20	TCP	2962	80 → 58848 [PSH, ACK] Seq=13217 Ack=153 Win=65536 Len=2896 TSval=316031846 TSecr=1624218634 [TCP PDU reassembled in 500]
17	0.070792	172.20.0.10	172.20.0.10	TCP	66	58848 → 80 [ACK] Seq=153 Ack=3081 Win=78656 Len=0 TSval=1624218637 TSecr=316031830
18	0.070838	172.20.0.10	172.20.0.20	TCP	2962	80 → 58848 [PSH, ACK] Seq=16113 Ack=153 Win=65536 Len=2896 TSval=316031848 TSecr=1624218637 [TCP PDU reassembled in 500]
19	0.070883	172.20.0.10	172.20.0.20	TCP	2962	80 → 58848 [PSH, ACK] Seq=19009 Ack=153 Win=65536 Len=2896 TSval=316031848 TSecr=1624218637 [TCP PDU reassembled in 500]
20	0.071399	172.20.0.10	172.20.0.10	TCP	66	58848 → 80 [ACK] Seq=153 Ack=5977 Win=78680 Len=0 TSval=1624218637 TSecr=316031838
21	0.071401	172.20.0.10	172.20.0.10	TCP	66	58848 → 80 [ACK] Seq=153 Ack=7425 Win=78972 Len=0 TSval=1624218637 TSecr=316031838
22	0.071448	172.20.0.10	172.20.0.20	TCP	2962	80 → 58848 [PSH, ACK] Seq=21905 Ack=153 Win=65536 Len=2896 TSval=316031848 TSecr=1624218637 [TCP PDU reassembled in 500]
23	0.071490	172.20.0.10	172.20.0.20	TCP	2962	80 → 58848 [PSH, ACK] Seq=24801 Ack=153 Win=65536 Len=2896 TSval=316031848 TSecr=1624218637 [TCP PDU reassembled in 500]
24	0.071963	172.20.0.10	172.20.0.10	TCP	66	58848 → 80 [ACK] Seq=153 Ack=18321 Win=84992 Len=0 TSval=1624218638 TSecr=316031839
25	0.071965	172.20.0.10	172.20.0.10	TCP	66	58848 → 80 [ACK] Seq=153 Ack=11769 Win=88064 Len=0 TSval=1624218638 TSecr=316031839
26	0.072004	172.20.0.10	172.20.0.20	TCP	4410	80 → 58848 [PSH, ACK] Seq=27697 Ack=153 Win=65536 Len=4344 TSval=316031849 TSecr=1624218638 [TCP PDU reassembled in 500]
27	0.072050	172.20.0.10	172.20.0.20	TCP	4410	80 → 58848 [PSH, ACK] Seq=32041 Ack=153 Win=65536 Len=4344 TSval=316031849 TSecr=1624218638 [TCP PDU reassembled in 500]
28	0.072529	172.20.0.10	172.20.0.10	TCP	66	58848 → 80 [ACK] Seq=153 Ack=13217 Win=91136 Len=0 TSval=1624218638 TSecr=316031839
29	0.072633	172.20.0.10	172.20.0.20	TCP	4410	80 → 58848 [PSH, ACK] Seq=36385 Ack=153 Win=65536 Len=4344 TSval=316031850 TSecr=1624218638 [TCP PDU reassembled in 500]
30	0.072674	172.20.0.10	172.20.0.20	TCP	1514	80 → 58848 [ACK] Seq=48729 Ack=153 Win=65536 Len=1448 TSval=316031850 TSecr=1624218638 [TCP PDU reassembled in 500]
31	0.072753	172.20.0.10	172.20.0.10	TCP	66	58848 → 80 [ACK] Seq=153 Ack=16113 Win=90256 Len=0 TSval=1624218645 TSecr=316031846
32	0.072792	172.20.0.10	172.20.0.20	TCP	5858	80 → 58848 [PSH, ACK] Seq=42177 Ack=153 Win=65536 Len=5792 TSval=316031856 TSecr=1624218645 [TCP PDU reassembled in 500]
33	0.080927	172.20.0.10	172.20.0.10	TCP	66	58848 → 80 [ACK] Seq=153 Ack=19009 Win=96256 Len=0 TSval=1624218647 TSecr=316031848
34	0.080929	172.20.0.10	172.20.0.10	TCP	66	58848 → 80 [ACK] Seq=153 Ack=21905 Win=94208 Len=0 TSval=1624218647 TSecr=316031848
35	0.080976	172.20.0.10	172.20.0.20	TCP	5858	80 → 58848 [PSH, ACK] Seq=47969 Ack=153 Win=65536 Len=5792 TSval=316031858 TSecr=1624218647 [TCP PDU reassembled in 500]
36	0.081596	172.20.0.10	172.20.0.10	TCP	66	58848 → 80 [ACK] Seq=153 Ack=24801 Win=96256 Len=0 TSval=1624218647 TSecr=316031848
37	0.081599	172.20.0.10	172.20.0.10	TCP	66	58848 → 80 [ACK] Seq=153 Ack=27697 Win=94208 Len=0 TSval=1624218647 TSecr=316031848
38	0.081643	172.20.0.10	172.20.0.20	TCP	7306	80 → 58848 [PSH, ACK] Seq=53761 Ack=153 Win=65536 Len=7240 TSval=316031859 TSecr=1624218647 [TCP PDU reassembled in 500]
39	0.081693	172.20.0.10	172.20.0.10	TCP	7306	80 → 58848 [PSH, ACK] Seq=61001 Ack=153 Win=65536 Len=7240 TSval=316031859 TSecr=1624218647 [TCP PDU reassembled in 500]
40	0.082086	172.20.0.10	172.20.0.10	TCP	66	58848 → 80 [ACK] Seq=153 Ack=32041 Win=95232 Len=0 TSval=1624218648 TSecr=316031849
41	0.082088	172.20.0.10	172.20.0.10	TCP	66	58848 → 80 [ACK] Seq=153 Ack=36385 Win=91136 Len=0 TSval=1624218648 TSecr=316031849
42	0.082126	172.20.0.10	172.20.0.20	TCP	8754	80 → 58848 [PSH, ACK] Seq=68241 Ack=153 Win=65536 Len=8688 TSval=316031859 TSecr=1624218648 [TCP PDU reassembled in 500]
43	0.082167	172.20.0.10	172.20.0.20	TCP	8754	80 → 58848 [PSH, ACK] Seq=76929 Ack=153 Win=65536 Len=8688 TSval=316031859 TSecr=1624218648 [TCP PDU reassembled in 500]
44	0.082703	172.20.0.10	172.20.0.10	TCP	66	58848 → 80 [ACK] Seq=153 Ack=48729 Win=95232 Len=0 TSval=1624218649 TSecr=316031859
45	0.082705	172.20.0.10	172.20.0.10	TCP	66	58848 → 80 [ACK] Seq=153 Ack=42177 Win=94208 Len=0 TSval=1624218649 TSecr=316031850
46	0.082743	172.20.0.10	172.20.0.20	TCP	8842	80 → 58848 [PSH, ACK] Seq=85617 Ack=153 Win=65536 Len=8776 TSval=316031860 TSecr=1624218649 [TCP PDU reassembled in 500]
47	0.082973	172.20.0.10	172.20.0.10	TCP	2962	80 → 58848 [PSH, ACK] Seq=94393 Ack=153 Win=65536 Len=2896 TSval=316031860 TSecr=1624218649 [TCP PDU reassembled in 500]
48	0.083910	172.20.0.10	172.20.0.20	TCP	1514	80 → 58848 [ACK] Seq=97289 Ack=153 Win=65536 Len=1448 TSval=316031861 TSecr=1624218649 [TCP PDU reassembled in 500]
49	0.088910	172.20.0.10	172.20.0.10	TCP	66	58848 → 80 [ACK] Seq=153 Ack=47969 Win=94208 Len=0 TSval=1624218655 TSecr=316031856
50	0.088994	172.20.0.10	172.20.0.20	TCP	11650	80 → 58848 [PSH, ACK] Seq=98737 Ack=153 Win=65536 Len=11584 TSval=316031866 TSecr=1624218655 [TCP PDU reassembled in 500]

Figura 10. Linha do tempo no Wireshark evidenciando a resposta DNS seguida imediatamente pelo SYN do TCP.

No.	Time	Source	Destination	Prot	Len	Info
1	0.000000	172.20.0.20	172.20.0.30	DNS	62	Standard query 0x2d52[Malformed Packet]
2	0.003000	172.20.0.30	172.20.0.20	DNS	66	Standard query response 0x2d52[Malformed Packet]
3	0.032000	172.20.0.20	172.20.0.30	Wi	171	Transport Data, receiver=0x57921900, counter=7364018928491692034, datalen=97

Figura 11. Transição estrita entre o encerramento da Query DNS e o primeiro datagrama de dados do R-UDP.

Em ambos os casos, a aplicação comporta-se de forma estritamente sequencial. O cliente dispara a *Standard Query* DNS em direção à porta 53 do servidor configurado e bloqueia a execução. No instante exato em que o pacote *Standard Query Response* é recebido pela interface de rede, o gatilho interno da aplicação realiza o *parsing* do endereço IP retornado e engata a transmissão de transporte. No fluxo clássico, isso reflete no envio imediato do pacote [SYN] para a porta HTTP padrão; no fluxo experimental, o evento dispara o primeiro pacote DATA sinalizado com a flag de inicialização do R-UDP.

5.3. Comparação Aplicação vs Wireshark

Para garantir a acurácia científica dos dados reportados, realizou-se uma validação cruzada entre os tempos marcados internamente pelos *logs* de *timestamp* da aplicação cliente e os tempos físicos de tráfego absoluto registrados pelo `tcpdump` na interface de rede.

**Tabela 3. Validação Cruzada de Métricas Temporais: Aplicação vs Wireshark
(Cenário A, 1 MB)**

Métrica / Camada	Tempo Aplicação (s)	Tempo Wireshark (s)	Discrepância Δ (ms)
Resolução DNS (TCP)	0,0039	0,0037	+0,20
Transferência HTTP/TCP	5,6385	5,6370	+1,50
Resolução DNS (R-UDP)	0,0035	0,0033	+0,20
Transferência HTTP/R-UDP	21,3152	21,3131	+2,10

Os resultados dispostos na Tabela 3 demonstram uma coerência rigorosa na coleta. A sutil discrepância positiva observada sistematicamente nas métricas capturadas pela aplicação ($\Delta \leq 2,10$ ms) não representa uma falha de amostragem, mas sim o reflexo físico inevitável do *overhead* de processamento do espaço de usuário (*user space*). Esse atraso infinitesimal engloba o tempo gasto pelo sistema operacional para realizar as chamadas de sistema (*syscalls* como *recvfrom* e *sendto*), copiar os buffers de dados do núcleo (*kernel space*) para a aplicação e executar o processamento lógico dos cabeçalhos. A proximidade quase exata dos valores chancela a confiabilidade empírica das rodadas de testes expostas neste relatório.

6. Discussão (Perguntas Obrigatórias)

Esta seção consolida a análise analítica do experimento, respondendo aos questionamentos estruturais com base nas evidências empíricas extraídas das capturas de pacotes e no comportamento observado das pilhas de protocolos sob estresse de rede.

6.1. Perda de Pacotes no DNS vs HTTP (Pergunta 1)

A introdução de perdas artificiais nos cenários B e C expôs uma assimetria fundamental entre a resiliência nativa da camada de transporte e a dependência de temporizadores na camada de aplicação. O protocolo DNS opera nativamente sobre UDP, um protocolo sem estado e desprovido de mecanismos internos de confirmação ou retransmissão. Consequentemente, quando um pacote de *Query* ou de *Response* DNS sofre descarte na rede, o protocolo de transporte não toma conhecimento do evento. A recuperação do fluxo fica estritamente a cargo de um *timeout* implementado no espaço de usuário pela própria aplicação cliente, que precisa estourar antes que uma nova tentativa síncrona de resolução seja disparada na porta 53. Esse comportamento justifica os massivos degraus de atraso observados nos gráficos temporais dos cenários hostis.

Em contrapartida, o protocolo HTTP encapsulado sobre a pilha experimental R-UDP demonstrou uma abordagem agressiva de tolerância a falhas para garantir a limpeza física de seus buffers. Diferente do UDP puro do DNS, o R-UDP foi projetado com um mecanismo estrito de até 7 tentativas iniciais de retransmissão por bloco. Sob condições de perda cumulativa, enquanto a aplicação aguarda passivamente o restabelecimento da resolução de nomes devido à ausência de inteligência no UDP nativo, a transmissão de dados no R-UDP força a persistência através dessas retransmissões consecutivas baseadas em temporizadores locais. Esse design garante que, uma vez superado o gargalo do DNS, o buffer de aplicação seja esvaziado e transmitido de forma confiável, ainda que penalizado severamente na métrica de vazão líquida (*throughput*) devido à natureza bloqueante do modelo *Stop-and-Wait*.

6.2. Overhead dos Cabeçalhos HTTP adicionados (Pergunta 2)

A evolução do experimento de um protocolo puramente textual e minimalista para uma especificação estruturada baseada em HTTP/1.1 introduziu um *overhead* métrico significativo que impactou diretamente a eficiência de canal. No modelo anterior, as mensagens trafegadas limitavam-se a comandos brutos diretos, consumindo poucos bytes de metadados por segmento. A introdução da semântica HTTP exigiu a inclusão de cabeçalhos textuais obrigatórios (como `Host`, `Content-Type`, `Content-Length` e `Connection`), além do campo customizado de auditoria de 64 bytes `X-Custom-Auth`.

Esse acréscimo de metadados textuais estendeu o tamanho físico da região de controle de cada datagrama. Visualmente, conforme auditado nos *dumps* hexadecimais do Wireshark, a proporção de carga útil (*goodput*) por pacote decresceu: uma quantidade notável de bytes que poderiam ser utilizados para transportar o arquivo binário de 1 MB passou a ser consumida para o transporte das *strings* de cabeçalho da camada de aplicação. No Cenário A (rede ideal), onde não há retransmissões, esse *overhead* traduziu-se em uma redução direta da taxa de transferência efetiva. A pilha precisou processar e transmitir um volume total de dados consideravelmente maior na rede para entregar o mesmo arquivo útil de 1 MB, evidenciando o preço estatístico pago pela padronização e segurança da arquitetura HTTP.

6.3. Sequência DNS → HTTP no Wireshark (Pergunta 3)

As evidências temporais registradas cronologicamente pelo `tcpdump` e inspecionadas no Wireshark comprovaram de forma irrefutável que a arquitetura real da rede seguiu rigorosamente a ordem sequencial esperada pelos modelos teóricos da Internet. Em todas as análises de fluxo efetuadas, a sincronia dos eventos manteve-se atômica e invariável.

O mapeamento milimétrico dos *timestamps* nas capturas demonstrou que o cliente jamais inicia qualquer procedimento de transporte antes da validação da camada de aplicação superior. O evento inicial é invariavelmente a saída da *Query* DNS ($t = 0,000$ s), seguida pela latência física de tráfego até o retorno do pacote de resposta do servidor DNS ($t = 0,003$ s). Somente após esse milissegundo exato, com a tradução do domínio consolidada na memória da aplicação, observa-se o disparo imediato do primeiro pacote de transporte (seja o `[SYN]` na porta TCP 80 ou o pacote inicial na porta R-UDP 5001). Essa dependência cronológica estrita prova empiricamente o acoplamento sequencial das camadas: o subsistema de transporte é completamente cego à semântica de nomes textuais, dependendo de forma absoluta do encerramento com sucesso da resolução de nomes para dar início à sessão de dados.

7. Conclusão

Este experimento permitiu consolidar, de forma prática e quantitativa, os impactos decorrentes da escolha de diferentes paradigmas de transporte e aplicação sobre o desempenho de redes sob condições de estresse. A migração da especificação simplificada para uma arquitetura padronizada em HTTP/1.1 introduziu os desafios reais de *overhead* estrutural e processamento semântico de cabeçalhos, aproximando o ambiente didático dos cenários industriais de engenharia de protocolos.

Os testes empíricos cancelaram o sucesso do protocolo experimental R-UDP no cumprimento de seu requisito primordial: garantir a confiabilidade na entrega de dados. Mesmo sob as taxas cumulativas de perda de pacotes e atrasos impostas nos cenários B e C, a integridade do arquivo binário de 1 MB foi preservada em todas as iterações, comprovando a robustez lógica de seus mecanismos de temporização, checagem de erros por CRC32 e controle de fluxo.

Contudo, a análise microscópica dos dados revelou limitações técnicas profundas inerentes à adoção da estratégia *Stop-and-Wait*. O protocolo mostrou-se altamente vulnerável à dessincronização de buffers nas etapas iniciais da transmissão. Como o modelo impõe um bloqueio síncrono que impede o envio de novos segmentos antes da confirmação inequívoca do bloco anterior, qualquer descarte de pacotes de controle ou de dados força o canal a um estado de completa ociosidade até o estouro estático do *timeout* em espaço de usuário. A ausência de uma janela deslizante dinâmica ou de algoritmos adaptativos de controle de congestionamento atuou como o principal gargalo técnico da pilha customizada, derrubando a vazão líquida (*throughput*) e expandindo massivamente o tempo de transferência total quando comparado à resiliência nativa do TCP e suas retransmissões rápidas.

Em suma, o trabalho evidenciou de forma incontestável os *trade-offs* que governam o design de protocolos. Enquanto o TCP reafirma sua eficiência através de décadas de otimizações adaptativas em nível de *kernel*, a implementação do R-UDP cumpriu seu papel acadêmico de expor detalhadamente a mecânica de controle de fluxo e os custos de engenharia necessários para forçar a confiabilidade sobre canais inerentemente não-confiáveis.

Referências

- Docker Inc. (2026). Docker documentation. Disponível em: <https://docs.docker.com>. Acesso em: maio 2026.
- Fielding, R. and Reschke, J. (2014). Hypertext transfer protocol (http/1.1): Message syntax and routing. RFC 7230, RFC Editor.
- Kurose, J. F. and Ross, K. W. (2021). *Redes de Computadores e a Internet: Uma Abordagem Top-Down*. Pearson, São Paulo, 8 edition.
- Linux Foundation (2026). tc-netem: Network emulator. Disponível em: <https://man7.org/linux/man-pages/man8/tc-netem.8.html>. Acesso em: maio 2026.
- Mockapetris, P. (1987). Domain names - concepts and facilities. RFC 1034, RFC Editor.
- Postel, J. (1980). User datagram protocol. RFC 768, RFC Editor.
- Postel, J. (1981). Transmission control protocol. RFC 793, RFC Editor.
- Tanenbaum, A. S. and Wetherall, D. (2011). *Redes de Computadores*. Pearson, São Paulo, 5 edition.